~/majibill/docs/05-devops-infrastructure.md

$ cat 05-devops-infrastructure.md

MAJIBILL PLATFORM · DEVOPS

DevOps & Infrastructure

Hosting options in Kenya, cell topology, CI/CD, environments, observability, reliability, cost model and sovereign installs.

```
terraform apply -var cell=nbo-silo-01
# one artifact, three kinds of cell
# RPO 5 min / RTO 1 h for the enterprise cell
```

DOC
MB-05

VERSION
1.0 draft

DATE
08 Oct 2026

STATUS
For review

AUDIENCE
Platform/DevOps engineers, tech lead, finance

CLASS
Confidential

Prepared by 404HubSpot · Nakuru, Kenya · https://404-hub-spot.vercel.app
Working codename: MajiBill (Swahili: maji = water). Figures marked 'assumption' must be validated before commitment.

// Contents

1. Goals and constraints . . . 3
2. Where to host . . . 3
3. Reference topology of one cell . . . 4
4. CI/CD . . . 5
5. Infrastructure as code . . . 5
6. Observability and operations . . . 6
7. Cost model (planning estimates) . . . 7
8. Sovereign (client-owned) installs . . . 8

// 1. Goals and constraints

Run a revenue system for utilities reliably with a small team and a startup budget, from a single pilot to a Nairobi-scale tenant, while keeping a credible path to client-owned installs.

Constraint	Implication
Small team, agent-assisted	Managed services over self-run; boring, well-documented tooling; everything as code
Startup budget	Start with a lean pilot cell; scale by measured need; reserve spend for backups, monitoring, and security, never skip those
Connectivity in Kenya	Edge caching for the web apps, lightweight payloads, SMS fallback; offline-first mobile; avoid chatty APIs
Data protection law	Know where data lives; encrypted backups; ability to host in Kenya if a client requires it (MB-07)
Money flow	RPO measured in minutes; tested restores; idempotent processing so recovery is safe
Sovereign option	Containers + Terraform + compose/Helm; no hard dependency on one cloud's proprietary services

// 2. Where to host

Verify current regions, pricing and in-country options before committing; this table reflects the decision logic, and provider offerings change.

Option	Pros	Cons	Use for
AWS af-south-1 (Cape Town)	Mature managed Postgres, ECS, WAF, KMS; strong tooling; Africa region with reasonable latency to Nairobi	Not in Kenya; prices higher than major regions	**Default SaaS cells**
Azure South Africa / GCP Johannesburg	Comparable managed services	Same residency caveat	Alternative if a client mandates
Nairobi data centres (colocation / local cloud)	In-country data, lower latency, satisfies strict residency	More ops burden, fewer managed services	**Silo / sovereign cells** for clients that require in-country hosting
EU regions (cheapest)	Low cost	Higher latency, cross-border transfer obligations	Not recommended for production tenant data

> DECISION **Decision**
> Primary SaaS cells on AWS Cape Town with encrypted backups. Offer an in-country silo cell (Nairobi colocation or local cloud) as an enterprise option. Cross-border backup copies are a per-tenant decision that must be agreed in the data processing agreement.

// 3. Reference topology of one cell

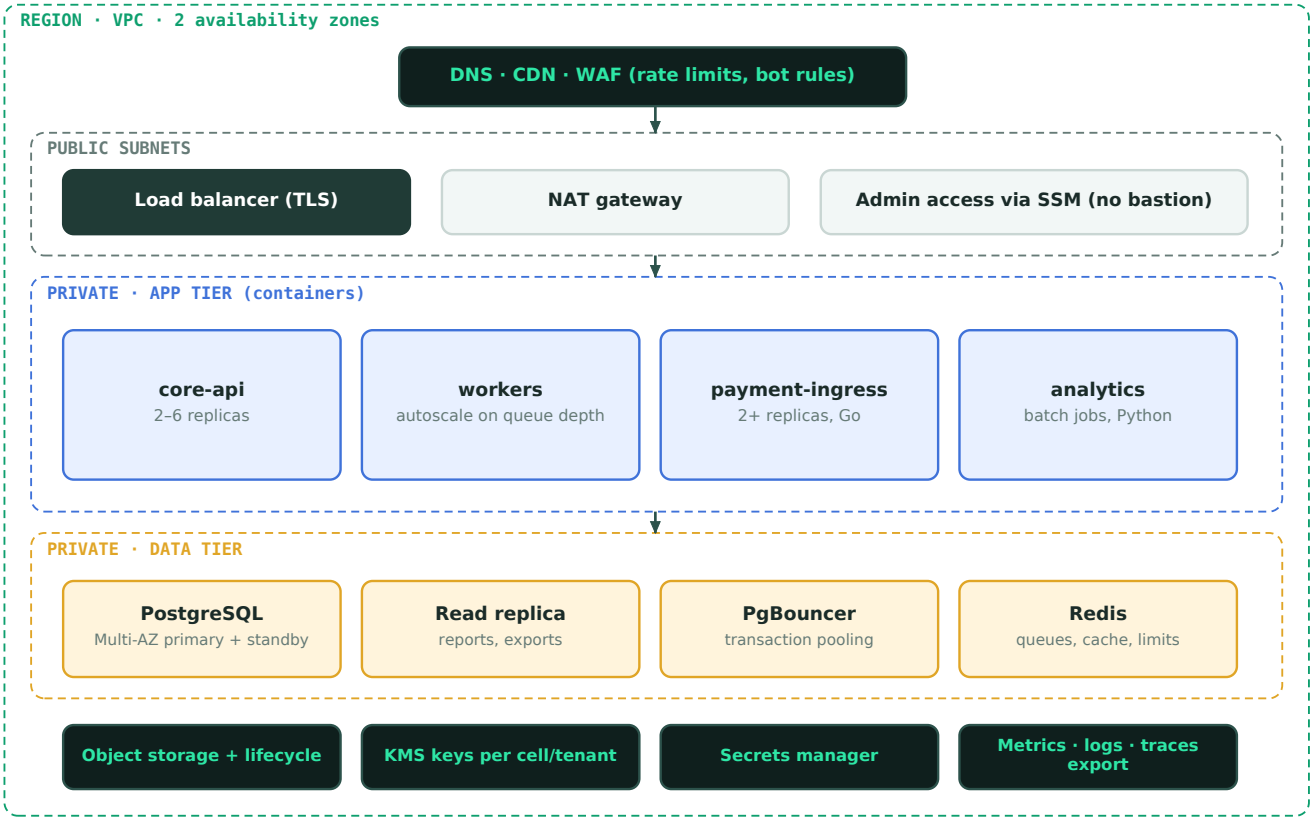


Fig 3.1 One cell: public edge, private application tier, private data tier, shared managed services. A silo cell is the same diagram with one tenant.

3.1 Environments

Environment	Purpose	Data	Deploy
Local	Developer + agent inner loop	Synthetic seed data	Docker Compose
Preview (per PR)	Review UI and API changes	Synthetic, ephemeral	Automatic on PR
Staging	Release candidate, load and sync tests, client demos	Anonymised/synthetic at scale	Automatic on merge to main
Prod: pooled cell(s)	Small tenants	Real	Gated, canary
Prod: silo cell(s)	Large or special tenants	Real	Gated, canary, tenant maintenance window

Production data never flows to staging unmasked. Seed generators produce realistic Kenyan-style names, phone numbers (E.164), account numbers and tariff structures.

// 4. CI/CD

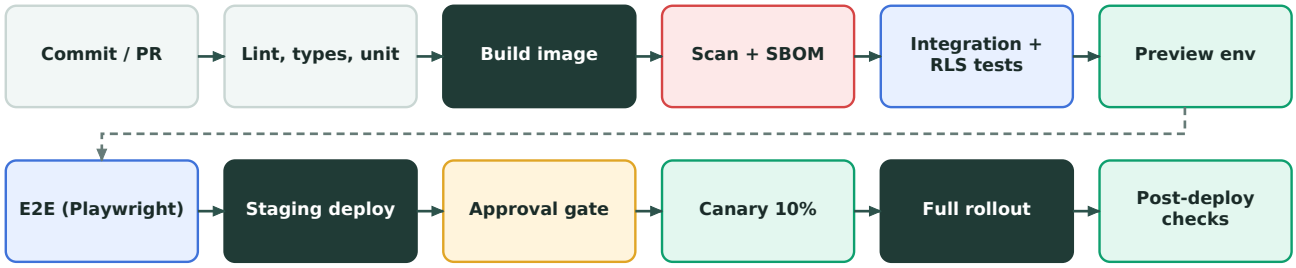


Fig 4.1 Pipeline from commit to production. Failing any gate stops the line; rollbacks are one command.

Practice	Detail
Branching	Trunk-based; short-lived branches; feature flags (OpenFeature-compatible) for incomplete work
Cloud credentials	GitHub OIDC to cloud IAM roles; no long-lived keys in CI
Database migrations	Run as a separate pre-deploy job with `lock_timeout`; **expand/contract** so old and new app versions both work; destructive steps a release later
Release cadence	Weekly release train for tenants; hotfix path with expedited approval; money-code changes need two reviewers
Rollback	Previous image + flag off; migrations are forward-only so we design them to be safe with the previous version
Supply chain	Dependabot, Trivy image scans, SBOM (CycloneDX), signed images, pinned base images, secret scanning
Quality gates	Coverage thresholds on domain code, mutation tests on billing, OpenAPI diff check, bundle-size budget for web, CodeRabbit review

// 5. Infrastructure as code

terraform layout

```
infra/terraform/
  modules/
    network/        # VPC, subnets, endpoints
    cell/           # compose of services below, parameterised by kind=pooled|silo
    database/       # Postgres, parameter group, PITR, replicas
    app/            # container services, autoscaling, alarms
    observability/  # dashboards, alarms, log retention
    tenant-bootstrap/ # registry entry, first admin, RLS smoke test
  envs/ staging/  prod-pooled-01/  prod-silo-nbo-01/
```

- Remote state with locking; plan on PR, apply on merge to protected environments.
- Policy-as-code (Checkov/tfsec) in CI; drift detection nightly.
- A new cell is `terraform apply` + a registry entry; a new tenant is an API call (`tenant-bootstrap`).
- Same modules can target a Nairobi colocation via a different provider block, or output a compose bundle for sovereign installs.

// 6. Observability and operations

Signal	Standard
Logs	Structured JSON; fields `tenant_id`, `request_id`, `trace_id`, `actor_id`; **no PII, no secrets**; 30–90 day retention hot, archived cold
Metrics	RED for services (rate, errors, duration), USE for infra; business metrics: bills issued, payments processed, suspense count, queue lag
Traces	OpenTelemetry across API, workers, Go ingress; sampled; trace id returned to clients in errors
Errors	Sentry for web, API, mobile; release tracking; PII scrubbing
Dashboards	Fleet view (all cells), cell view, tenant view (for support), money-path view
Uptime	External synthetic checks from Nairobi and Europe: login, payment callback health, portal

6.1 SLOs and alerts

SLO / alert	Threshold	Page?
API availability	99.9% monthly; burn-rate alerts at 2% / 5% of budget	Yes
Payment ingress ack time	p99 under 500 ms; any 5xx burst	Yes
Payment confirm → ledger lag	Over 5 min for any payment	Yes
Suspense queue age	Items older than 24 h	Ticket
Bill run progress	Behind plan by 30%, or failed routes	Yes (run window)
Queue depth / DLQ	DLQ not empty; depth growing 15 min	Ticket / Yes
DB health	Replication lag over 60 s, connections over 80%, storage over 75%	Yes
Backups	Missed snapshot or WAL gap; restore drill overdue	Yes
Cert / secret expiry	Under 21 days	Ticket

6.2 Reliability and disaster recovery

Topic	Standard
Backups	WAL archiving + PITR; daily snapshots; encrypted; cross-region or in-country copy per tenant agreement
Targets	Enterprise cell: RPO 5 min, RTO 1 h. Pooled cell: RPO 15 min, RTO 4 h
Restore drills	Quarterly full restore to an isolated environment; timed and recorded; checks ledger invariants after restore
High availability	Multi-AZ database, 2+ replicas of each stateless service, health-checked rolling deploys
Game days	Twice a year: kill primary, saturate queue, corrupt a callback, lose a worker pool
Runbooks	Failover, restore, replay payments from inbox, stuck bill run, DLQ triage, rotate secrets, tenant migration A→B
On-call	Rotation of 2–3 engineers; incident commander role; blameless reviews within 5 working days

// 7. Cost model (planning estimates)

USD per month, **assumptions to verify with provider calculators** for the chosen region. SMS and M-Pesa fees are pass-through to tenants and excluded.

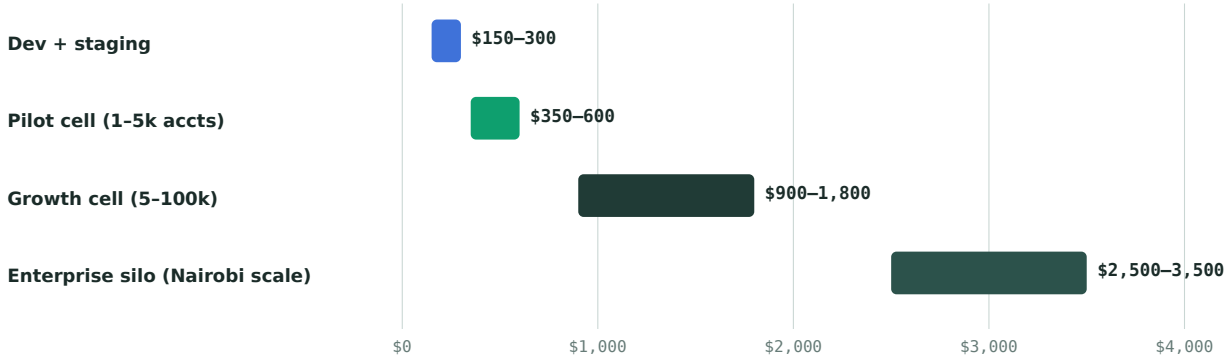


Fig 7.1 Planning ranges by cell tier (USD/month, excluding pass-through SMS and payment fees).

Component (enterprise cell)	Assumption	USD / month (range)
App tier containers	4 API, 4 workers, 2 ingress, burst batch jobs	350–600
PostgreSQL Multi-AZ + storage + PITR	4–8 vCPU class, 1 TB growing	900–1,300
Read replica	reports and exports	400–550
Redis, load balancer, WAF, NAT, transfer	managed, 2 AZ	450–650
Object storage + backups + KMS/secrets	documents, evidence, WAL, snapshots	150–250
Observability (Grafana stack / SaaS)	logs, metrics, traces, Sentry	250–450

Cost levers

- Right-size after one real bill-run cycle; use spot/burst capacity for batch workers only.
- Savings plans or reservations only after 3 months of stable usage.
- Log sampling and retention tiers; store raw callbacks compressed after 90 days.
- Pooled cells absorb many small tenants cheaply; do not give every small tenant a silo.
- SMS is the largest variable cost. As an illustration, 750k notices at an assumed KES 0.80 is about KES 600k per month, so price it separately or cap it per plan.

// 8. Sovereign (client-owned) installs

If a client buys the system, the same release ships as a **bundle**: signed container images, a compose file (small) or Helm chart (larger), migration job, backup scripts, runbooks, and a licence file. Updates come via a signed release channel the client can pull on their schedule.

Install size	Typical footprint	Notes
Small (up to ~20k accounts)	1 VM (4 vCPU, 16 GB, 200 GB SSD) running compose: API, workers, ingress, Postgres, Redis	Nightly backups to client storage + off-box copy; UPS and monitoring advised
Medium (up to ~150k)	3 VMs: app, database primary, database replica/backup; external object storage	Managed Postgres if available; reverse proxy with TLS
Large (Nairobi-scale)	Kubernetes or equivalent, HA Postgres, separate worker pool, observability stack	Use a managed or silo cell instead unless contractually required

8.1 DevOps maturity by phase

Phase	Must have	Can wait
Foundations (wk 1–3)	Repo, CI, IaC skeleton, staging, secrets, logging	Canary deploys, game days
Core build (wk 4–10)	Preview envs, migrations pipeline, base dashboards, backups with restore test	Multi-region DR
Pilot (wk 16–22)	Prod pooled cell, alerts + on-call, load test, runbooks, WAF, pen-test fixes	Silo automation
Scale-out (wk 23+)	Silo cell automation, tenant migration tooling, cost dashboards, DR drill	Kubernetes, sharding